

Unit-5

Curves and Fractals

- **Contents:**
- Curves: Introduction, Interpolation and Approximation, Blending function, B-Spline curve, Bezier curve,
- Fractals: Introduction, Classification, Fractal generation: snowflake, Triadic curve, Hilbert curve, Applications.

Curves & Fractals

- The object which are available in nature are generally having rough & curved surfaces.
- Fractals-
- Rough , jagged & random surfaces are called fractals.
- Example-mountains , trees , rivers , lighting a cloud, a flower so on.

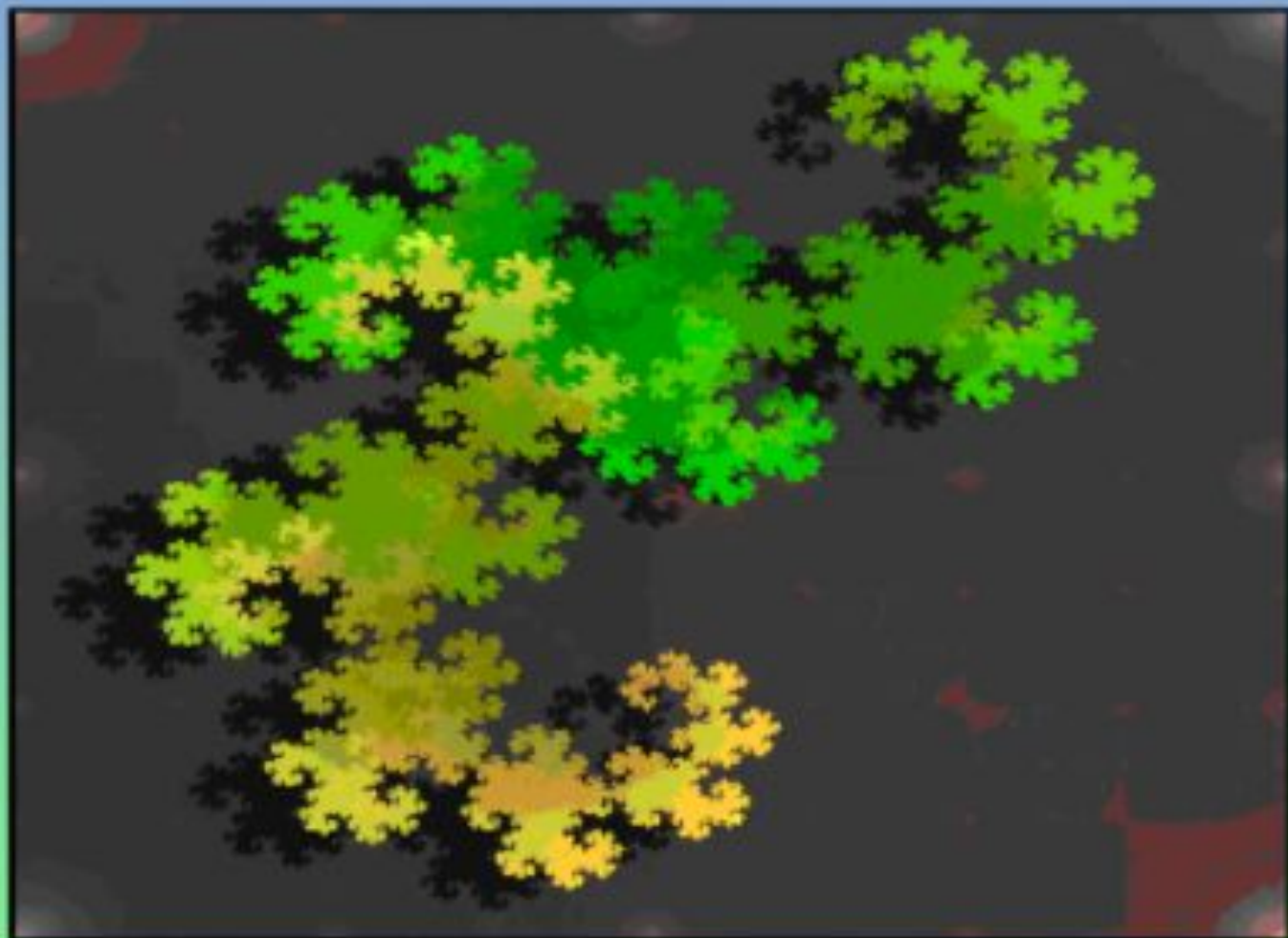
What are Fractals?

Fractals are very complex pictures generated by a computer from a single formula. They are created using iterations. This means one formula is repeated with slightly different values over and over again, taking into account the results from the previous iteration.

Fractals are used in many areas such as –

- **Astronomy** – For analyzing galaxies, rings of Saturn, etc.
- **Biology/Chemistry** – For depicting bacteria cultures, Chemical reactions, human anatomy, molecules, plants,
- **Others** – For depicting clouds, coastline and borderlines, data compression, diffusion, economy, fractal art, fractal music, landscapes, special effect, etc.

These fractals have a fixed geometric replacement rule.



Generation of Fractals

Fractals can be generated by repeating the same shape over and over again as shown in the following figure. In figure *a* shows an equilateral triangle. In figure *b*, we can see that the triangle is repeated to create a star-like shape. In figure *c*, we can see that the star shape in figure *b* is repeated again and again to create a new shape.

We can do unlimited number of iteration to create a desired shape. In programming terms, recursion is used to create such shapes.



(a) Zeroth Generation



(b) First Generation



(c) Second Generation



Figure: A fractal landscape

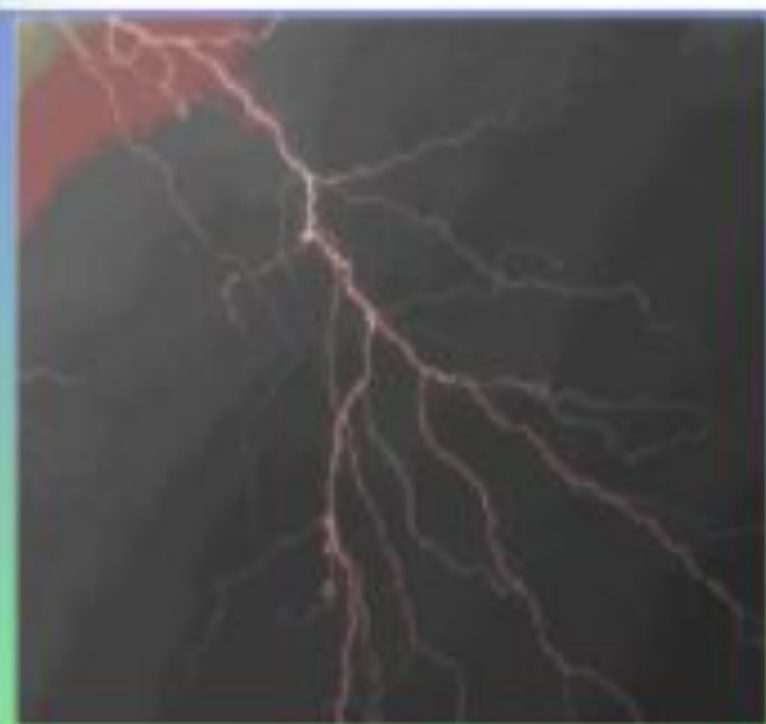
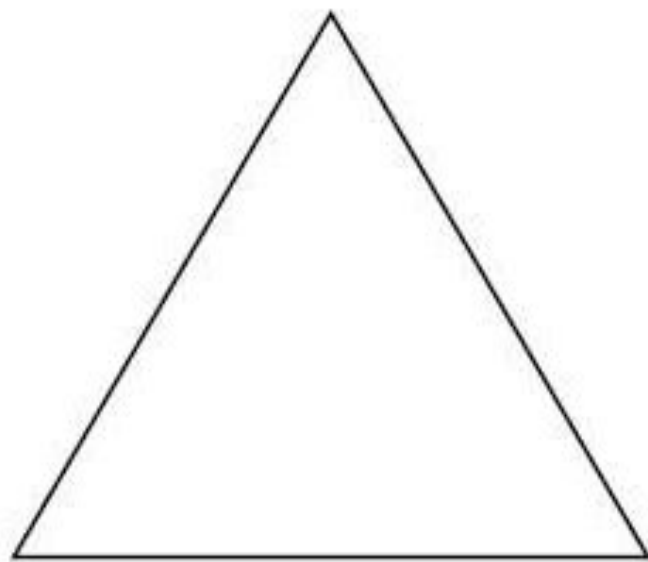


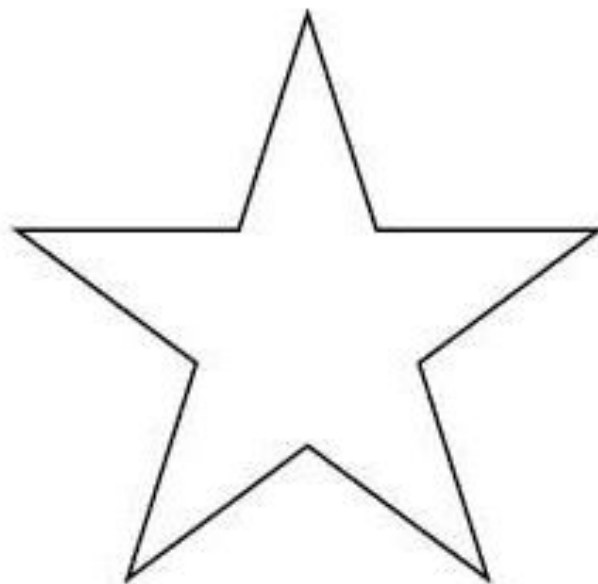
Figure: Lightning

Geometric Fractals

Geometric fractals deal with shapes found in nature that have non-integer or fractal dimensions. To geometrically construct a deterministic *nonrandom* self-similar fractal, we start with a given geometric shape, called the **initiator**. Subparts of the initiator are then replaced with a pattern, called the **generator**.



Initiator



Generator

As an example, if we use the initiator and generator shown in the above figure, we can construct good pattern by repeating it. Each straight-line segment in the initiator is replaced with four equal-length line segments at each step. The scaling factor is $1/3$, so the fractal dimension is $D = \ln 4 / \ln 3 \approx 1.2619$.

Also, the length of each line segment in the initiator increases by a factor of $4/3$ at each step, so that the length of the fractal curve tends to infinity as more detail is added to the curve as shown in the following figure –

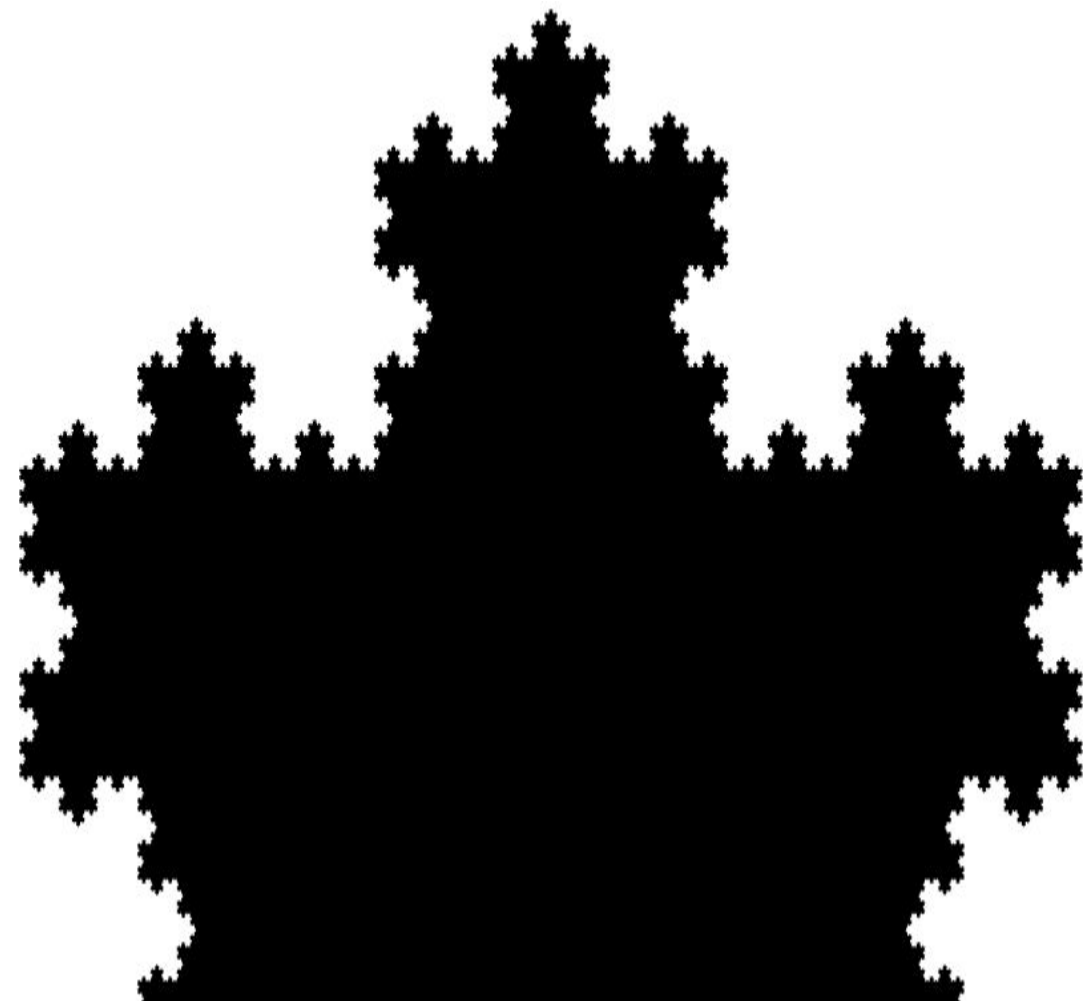
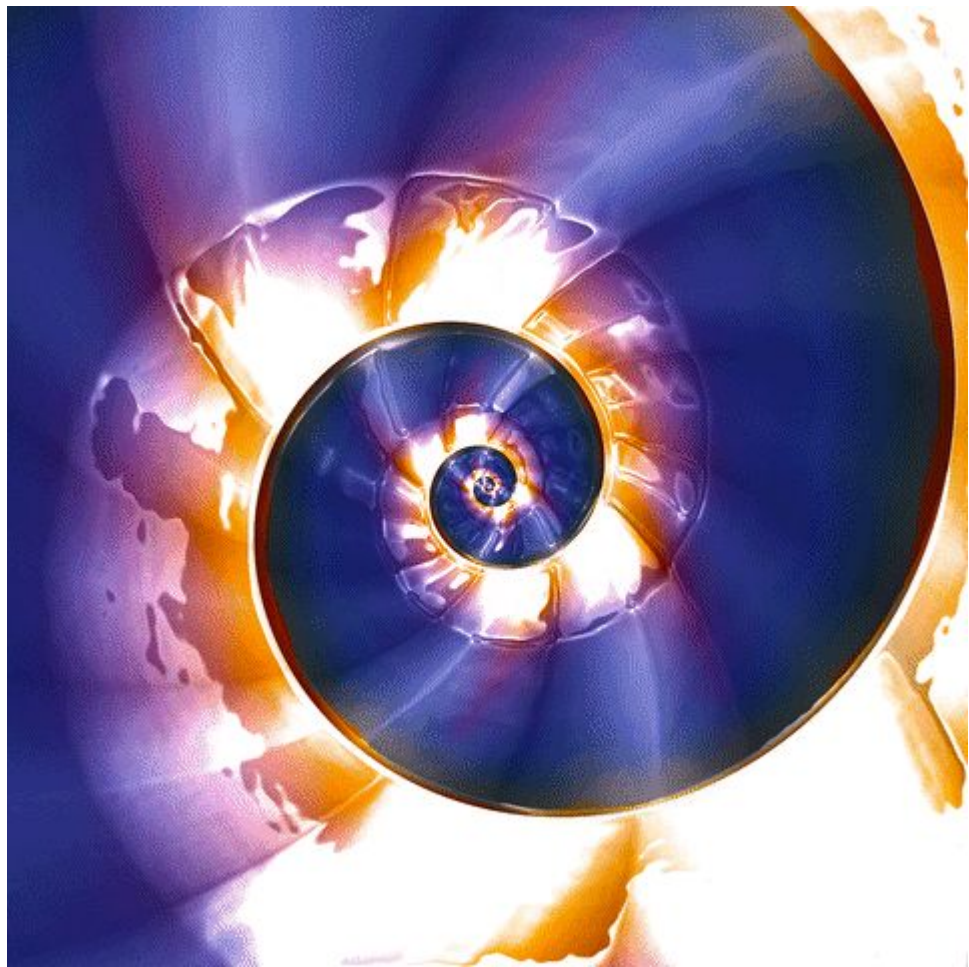
Segment Length = 1	Segment Length = $1/3$	Segment Length = $1/9$
		
Length = 1	Length = $4/3$	Length = $16/9$

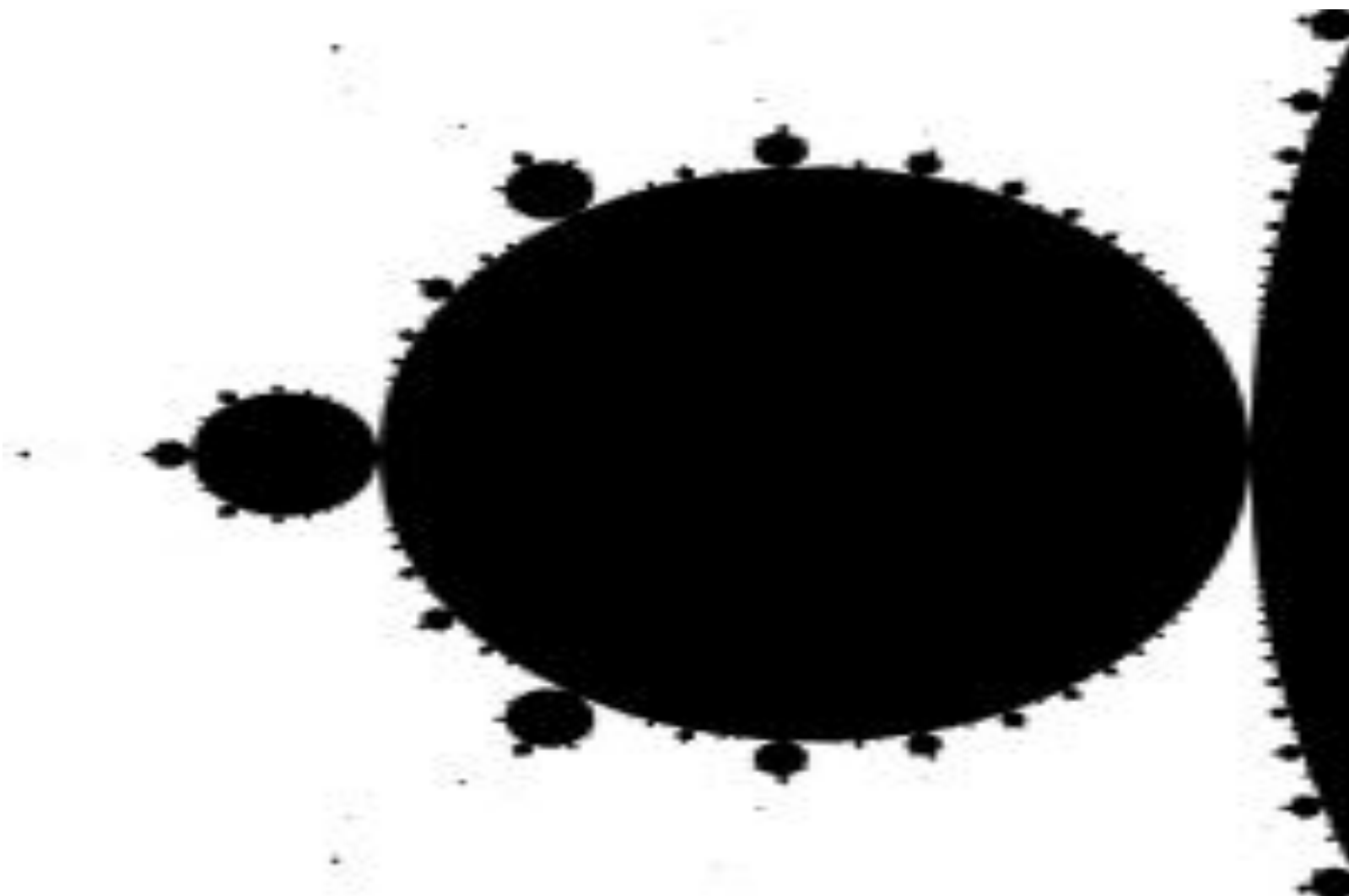
Classification of fractals

- Self similar
- Self affine
- Invariant

Self similar

- Fractals objects are constructed by applying a scaling parameter " s " to the overall initial shape.
- use same scale factor
- These fractals are commonly used to model trees , other plants.





- Self affine fractals-
 - subparts are formed with different scaling parameter s_x , s_y , s_z in different co-ordinate directions.
- Invariant fractals-
 - Non-linear transformation is used.
 - It includes self squaring fractals

Fractal dimension

- Assume a line segment of length L is divided into N identical pieces.
- So, the length of each line segment l can be given as,

$$l=L/N$$

but, ratio of length of original line segment & the length of each part of the line segment referred to as scaling factor is given as,

$$S=L/l$$

from above two equations we can write,

$$N=S$$

i.e. $N=S^1$

In case of $s=2$, we require 4 pieces of square to get original square.

$$N=s^2$$

Same as for cubical object we have

$$N=s^3$$

Dimension of the object by variable D .

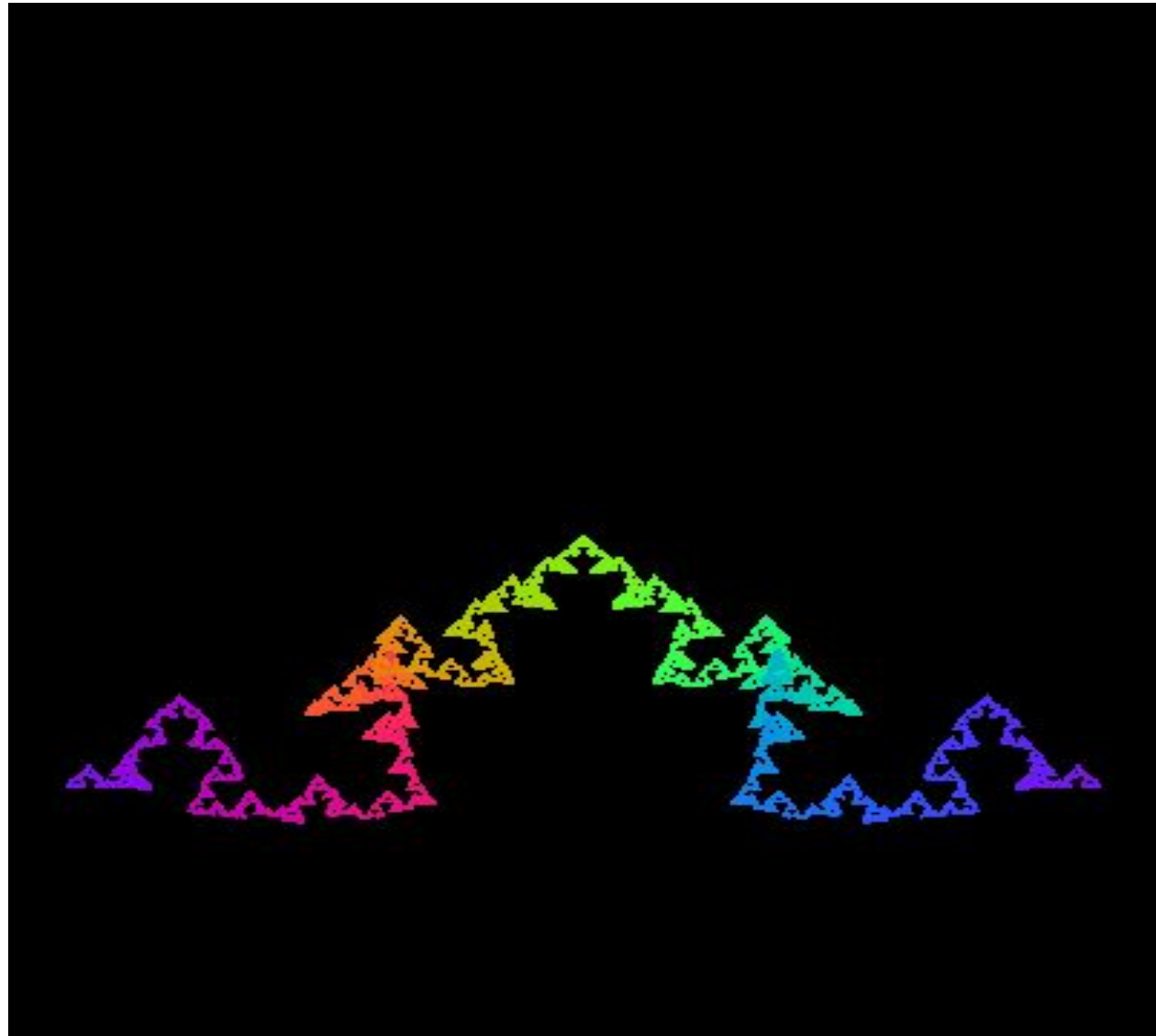
$$N=S^D$$

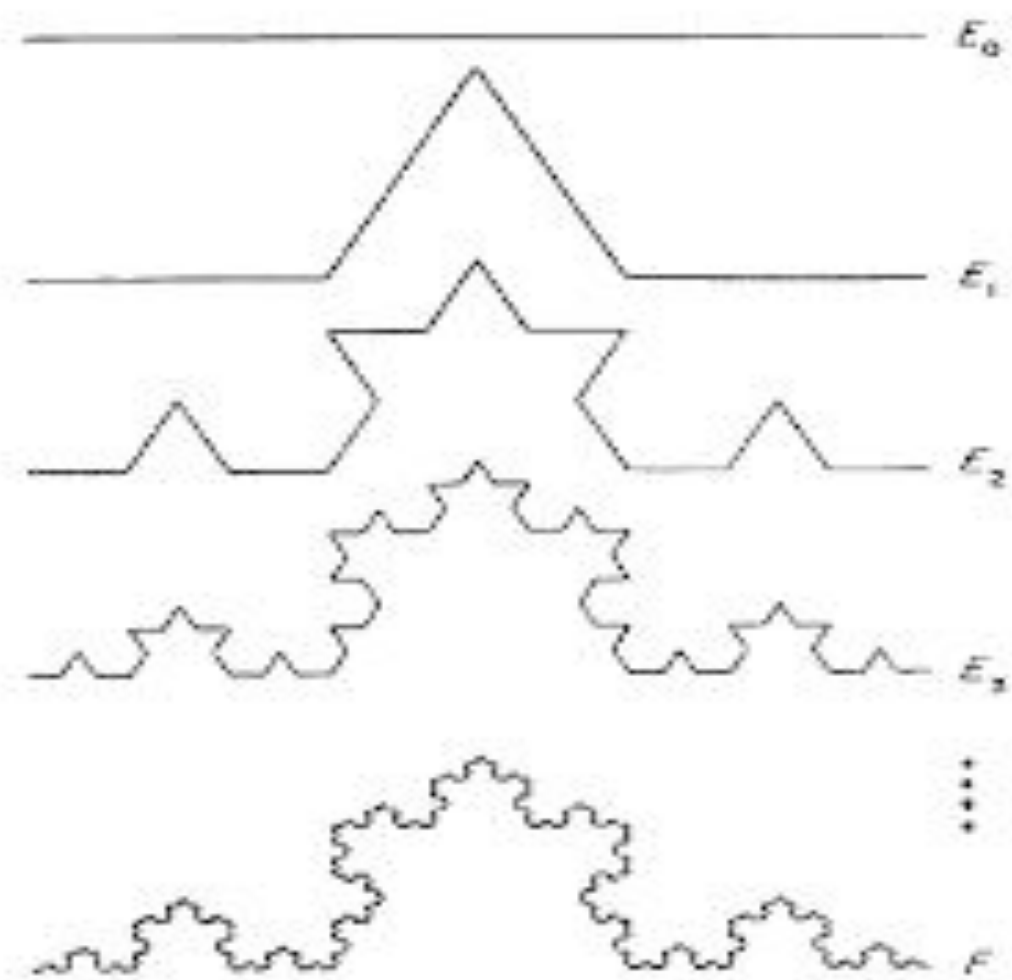
- Solving for D we get, $D=\log N / \log S$
- D is called as fractal dimension.

Cantor square



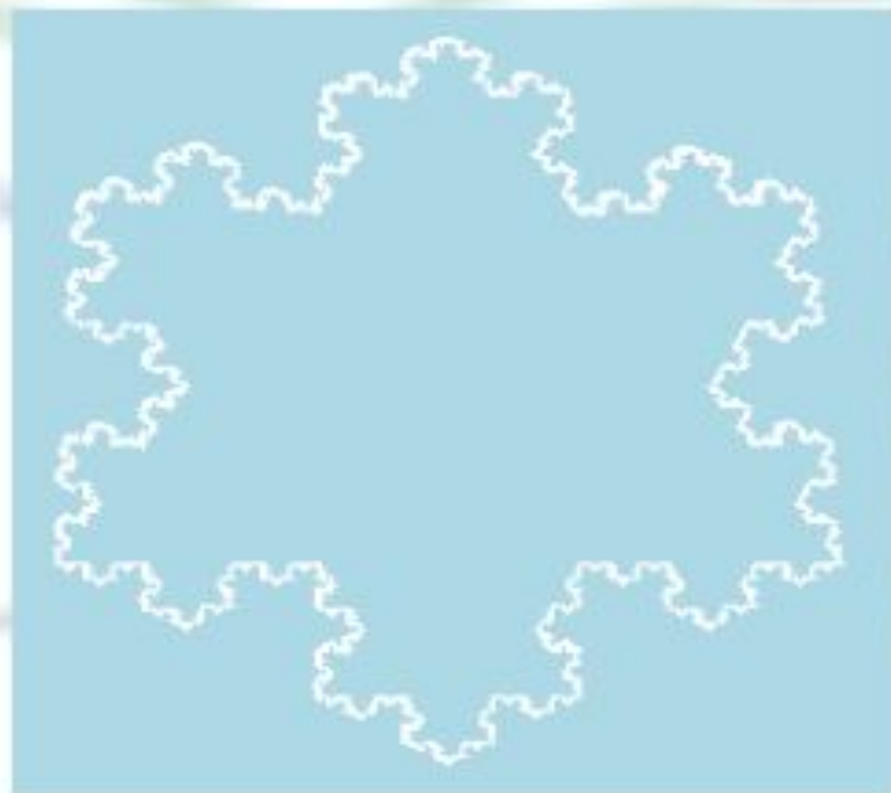
- Triadic Koch curve :- discovered by Helge von Koch.
- A straight line segment is called the first order.
- And then the four images above left are the next four orders.
- You can probably see how each order is built from the previous one.
- Koch curve is *very self-similar*.



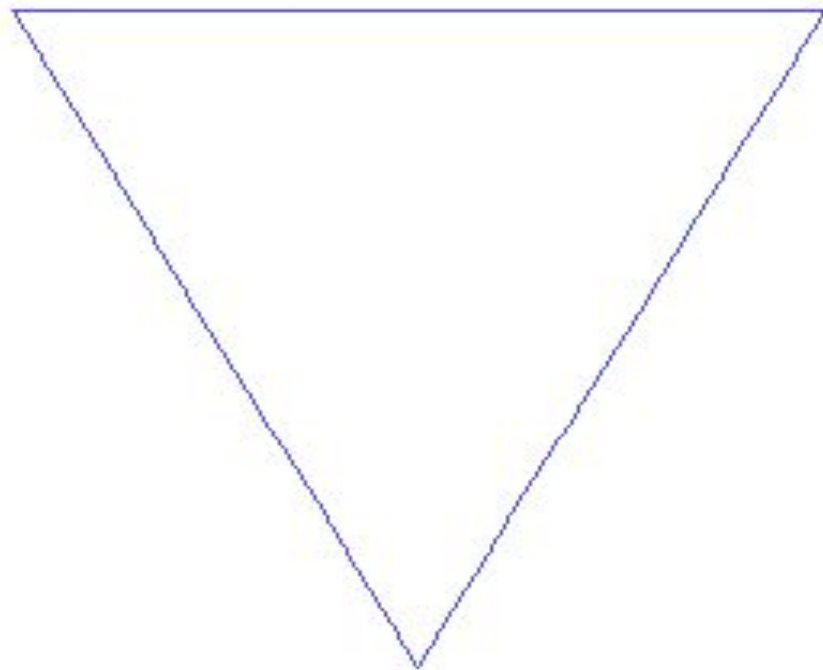


Snowflake

- Triadic koch curve procedure applied on triangle



Snowflake



Hilbert's Curve



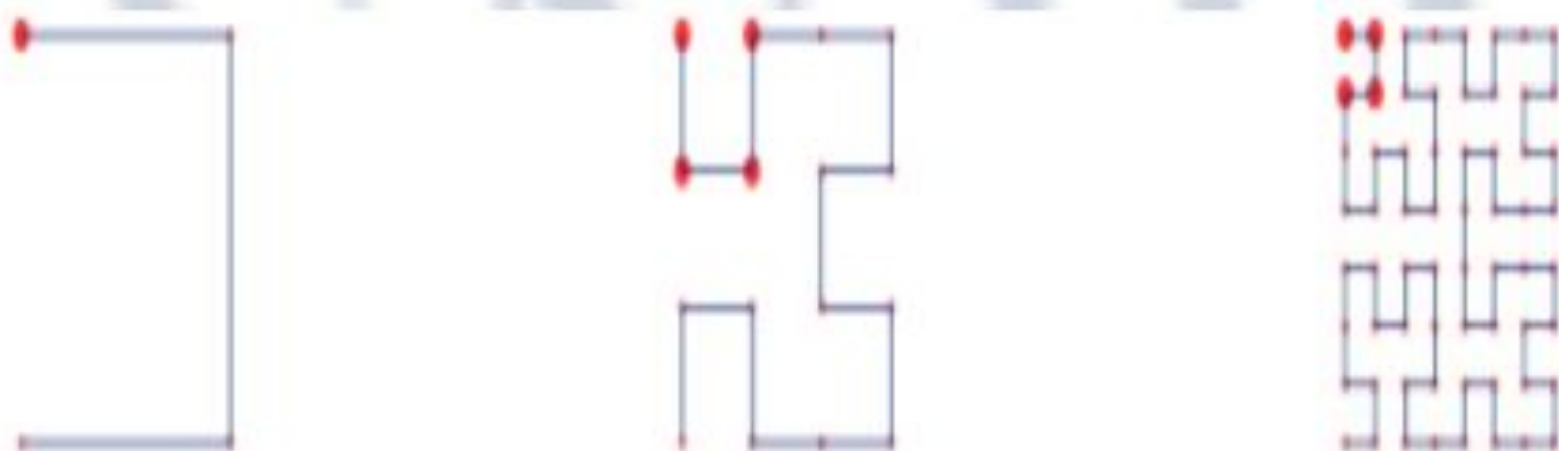
Hilbert curve, first order

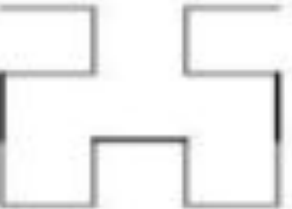
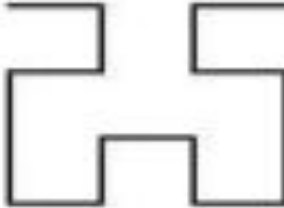
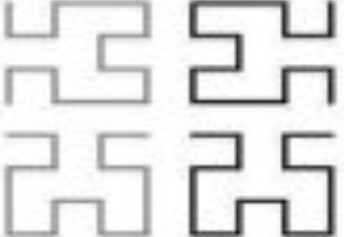
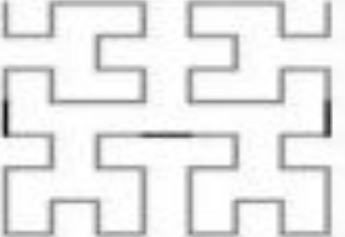
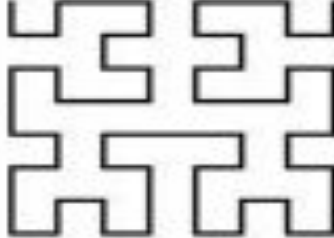
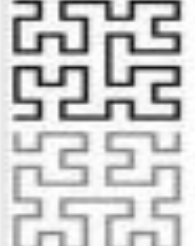
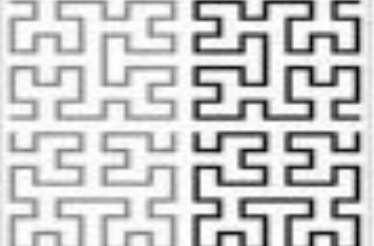
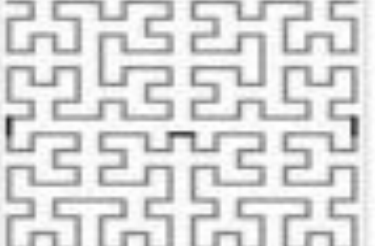
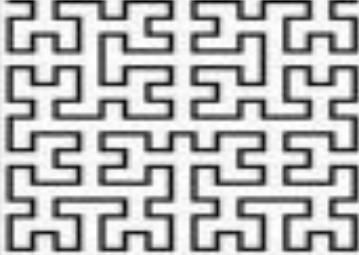
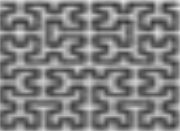
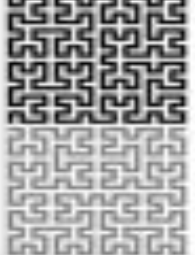
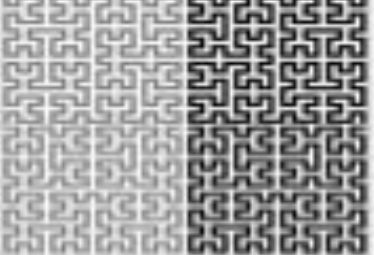
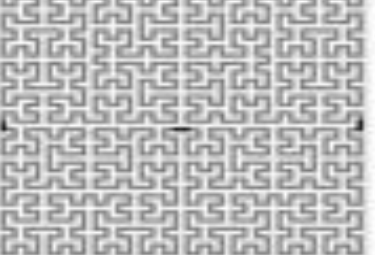


A Hilbert curve (also known as a Hilbert space-filling curve) is a continuous fractal space-filling curve first described by the German mathematician David Hilbert.

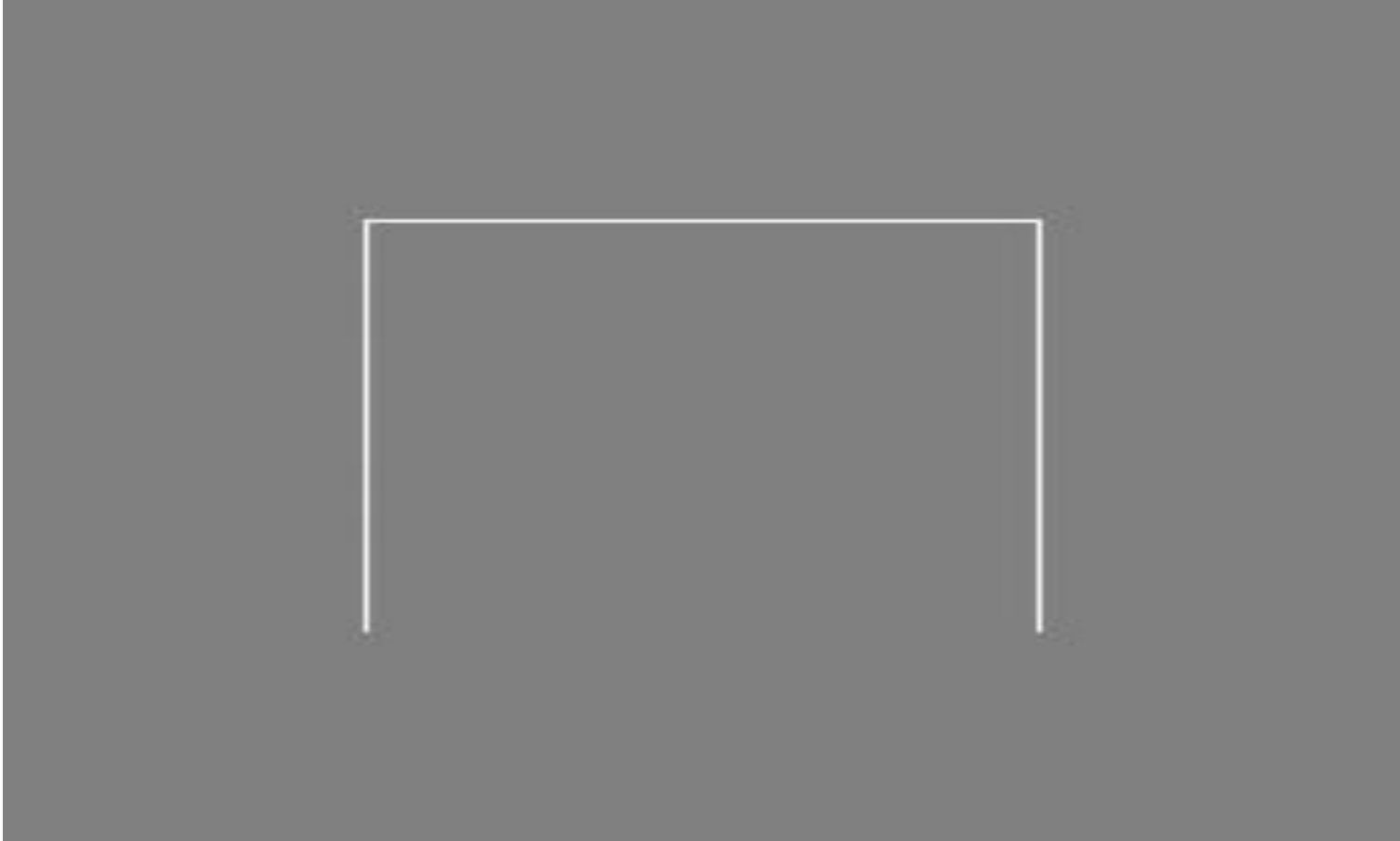


- we move from one order to the next, we replace each vertex with a sub-curve that has the same shape as the **O1** traversal.
- At every step, we also need to transform the subcurve through *rotation* and *reflection* to make sure that its start matches the end of the previous subcurve, and its end matches the beginning of the next subcurve.



	Starting curve	Transformation 1	Transformation 2	Transformation 3	Transformation 4
H_1					
H_2					
H_3					
H_4					

Hilbert Curve



Fractal Lines

- Fractal line can be generated by following steps:-
- If the straight line segment is specified by point (x_1, y_1, z_1) & (x_2, y_2, z_2) find the mid point of line by using following expression



$$((x_1+x_2)/2, (y_1+y_2)/2, (z_1+z_2)/2)$$

$$dx = L * W * G$$

$$dy = L * W * G$$

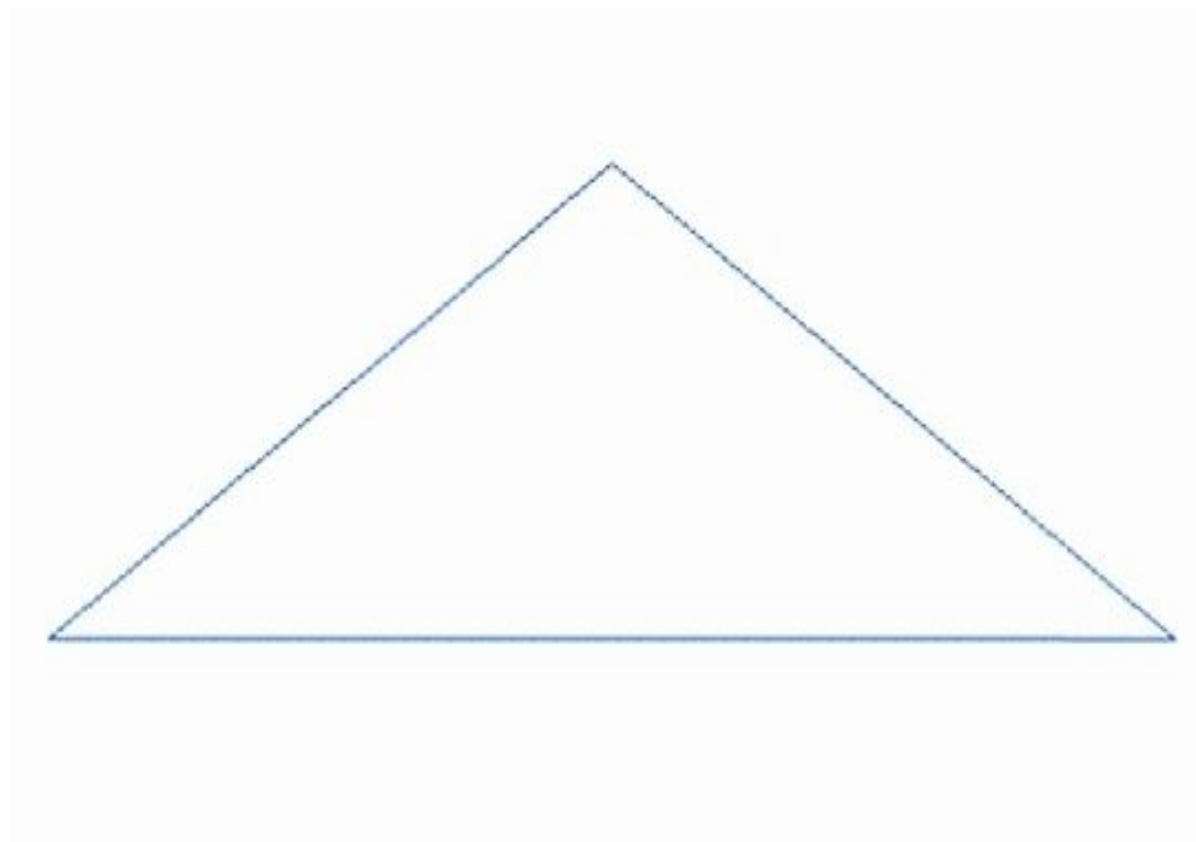
$$dz = L * W * G$$

- ▶ This midpoint divides the original line into two parts.
- ▶ Repeat the same process for each part separately.
- ▶ Repeat the process 1,2,3 until the segments become small enough.

The above implementation can easily be achieved using a recursive procedure.



Fractal surfaces



Curve

- Curve are one of the most essential objects to create high resolution graphics:
- While using small polylines allows creating graphics that appear smooth at fixed resolution, they do not preserve smoothness when scaled .
- Curve can be stored much easier, can be scaled to any resolution without losing smoothness and most importantly provide a much easier way to specify real world objects.

Types of Curves

A curve is an infinitely large set of points. Each point has two neighbors except endpoints.

Curves can be broadly classified into three categories – **explicit**, **implicit**, and **parametric curves**.

Implicit Curves

Implicit curve representations define the set of points on a curve by employing a procedure that can test to see if a point is on the curve. Usually, an implicit curve is defined by an implicit function of the form –

$$f(x, y) = 0$$

It can represent multivalued curves *multiple y values for an x value*. A common example is the circle, whose implicit representation is

$$x^2 + y^2 - R^2 = 0$$

Explicit Curves

A mathematical function $y = f(x)$ can be plotted as a curve. Such a function is the explicit representation of the curve. The explicit representation is not general, since it cannot represent vertical lines and is also single-valued. For each value of x , only a single value of y is normally computed by the function.

Parametric Curves

Curves having parametric form are called parametric curves. The explicit and implicit curve representations can be used only when the function is known. In practice the parametric curves are used. A two-dimensional parametric curve has the following form –

$$P(t) = f(t), g(t) \text{ or } P(t) = x(t), y(t)$$

The functions f and g become the x, y coordinates of any point on the curve, and the points are obtained when the parameter t is varied over a certain interval $[a, b]$, normally $[0, 1]$.

Curve generation method

- **DDA Circle drawing Algorithm**
- **Approximation/Interpolation method**
- We can draw an approximation to a curve if we have an array of sample points.
- The sample points are nothing but some points on the estimated path of the curve .
- From these few sample points, we can guess the shape of the curve .
- Our guess may not be exactly right but it can be acceptable one

Curve generation method



- There are many forms of the function, which we can make use by adjusting parameters to find the points between the sample points.

Curve generation method

- We are using parametric form

$$x=f_x(a)$$

$$y=f_y(a)$$

$$z=f_z(a)$$

As this curve is passing from n sample points the function must be sum of the n terms

$$f_x(a)=\sum x_i B_i(a) \text{ from } i=1 \text{ to } n$$

$$f_y(a)=\sum y_i B_i(a) \text{ from } i=1 \text{ to } n$$

$$f_z(a)=\sum z_i B_i(a) \text{ from } i=1 \text{ to } n$$

Where for each function we are using one common function $B_i(a)$ is called as Blending Function.

Curve generation method

- Blending Function:-
- To achieve a perfect curve it must pass through all the sample points. It means each sample point tries to pull the curve in his direction.
- If curve pass through sample point then $a=1$ otherwise 0

Curve generation method

- Blending Function:-
- Expression for the curve passing through sampling points
- $x = x_1 B_1(a) + x_2 B_2(a) + x_3 B_3(a) + x_4 B_4(a)$
- $Y = y_1 B_1(a) + y_2 B_2(a) + y_3 B_3(a) + y_4 B_4(a)$
- $Z = z_1 B_1(a) + z_2 B_2(a) + z_3 B_3(a) + z_4 B_4(a)$
- By using blending function we can interpolate polygon also.

B-Spline Curve

- We will not try to force the curve to pass through the sample points but rather pull it into the neighborhood of the sample point.
- A set of blending functions which takes this approach is called B- splines .
- Basically splines means a strip which we have to move around the sample points.
- Generally the Bspline blending function were designed to eliminate sharp corners.

A B-spline curve is defined as a linear combination of control points P_i and B-spline basis function $N_{i,k}(t)$ given by

$$C(t) = \sum_{i=0}^n P_i N_{i,k}(t), \quad n \geq k - 1, \\ t \in [t_k - 1, t_n + 1]$$

Where,

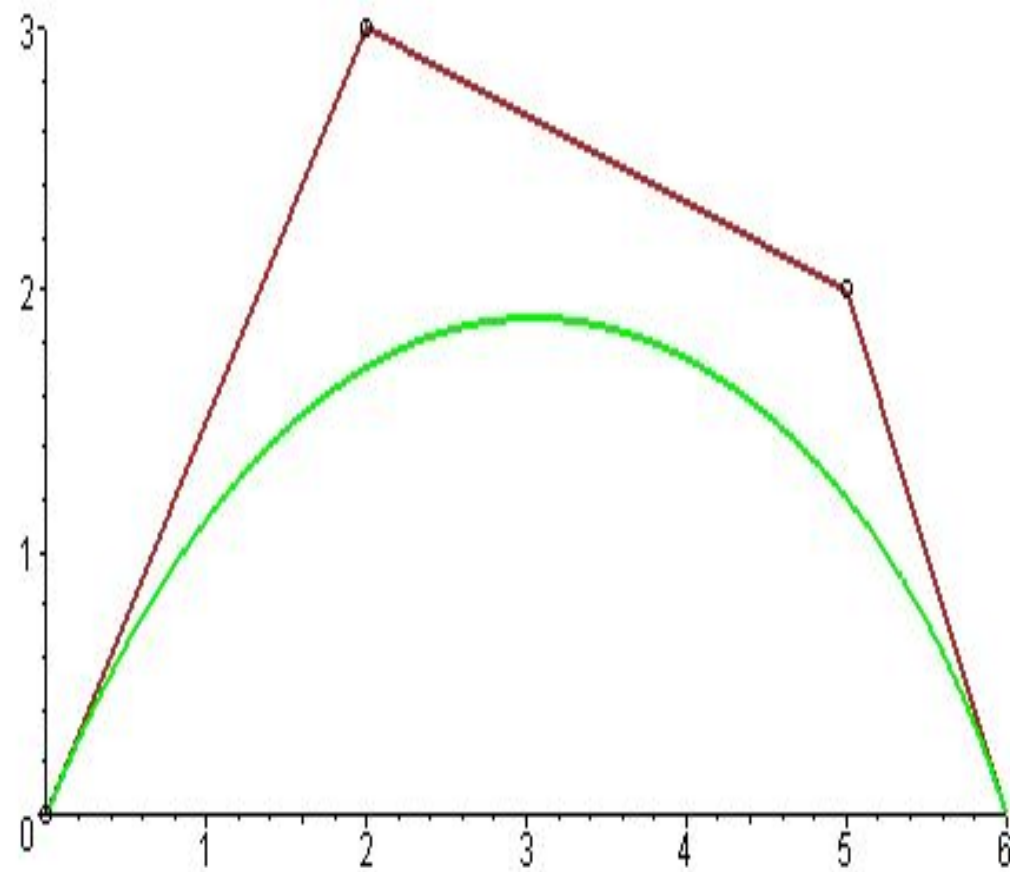
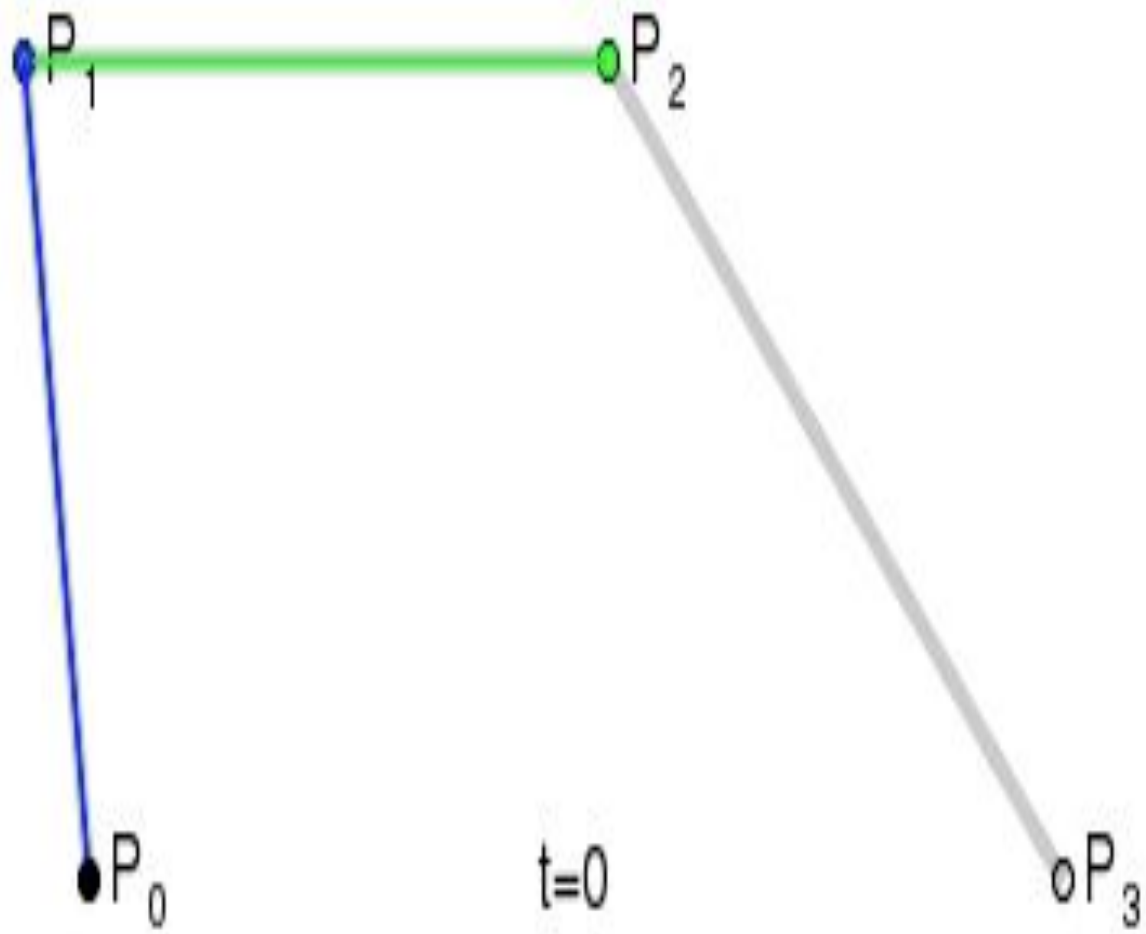
- $\{p_i : i=0, 1, 2, \dots, n\}$ are the control points
- k is the order of the polynomial segments of the B-spline curve. Order k means that the curve is made up of piecewise polynomial segments of degree $k - 1$,
- the $N_{i,k}(t)$ are the “normalized B-spline blending functions”. They are described by the order k and by a non-decreasing sequence of real numbers normally called the “knot sequence”.

$$t_i : i = 0, \dots, n + K$$

The $N_{i,k}$ functions are described as follows –

$$N_{i,1}(t) = \begin{cases} 1, & \text{if } t \in [t_i, t_{i+1}) \\ 0, & \text{Otherwise} \end{cases}$$

Bezier Curve



Bezier Curves

Bezier curve is discovered by the French engineer **Pierre Bézier**. These curves can be generated under the control of other points. Approximate tangents by using control points are used to generate curve. The Bezier curve can be represented mathematically as –

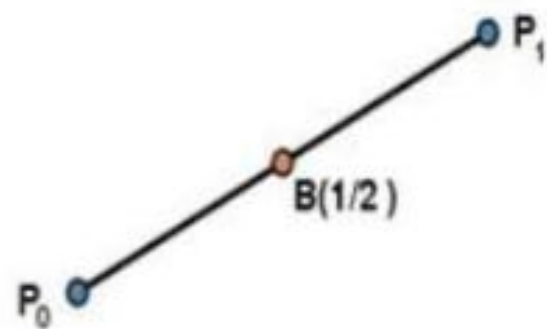
$$\sum_{k=0}^n P_i B_i^n(t)$$

Where p_i is the set of points and $B_i^n(t)$ represents the Bernstein polynomials which are given by –

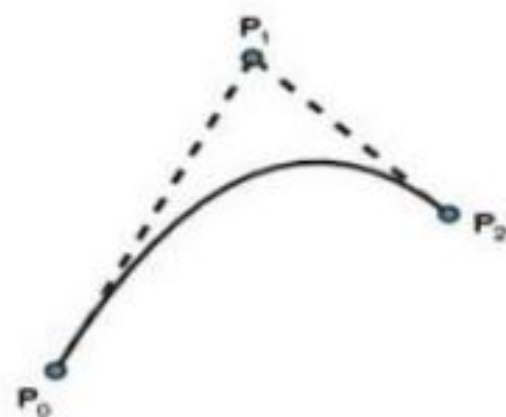
$$B_i^n(t) = \binom{n}{i} (1-t)^{n-i} t^i$$

Where **n** is the polynomial degree, **i** is the index, and **t** is the variable.

The simplest Bézier curve is the straight line from the point P_0 to P_1 . A quadratic Bezier curve is determined by three control points. A cubic Bezier curve is determined by four control points.



Simple Bezier Curve



Quadratic Bezier Curve



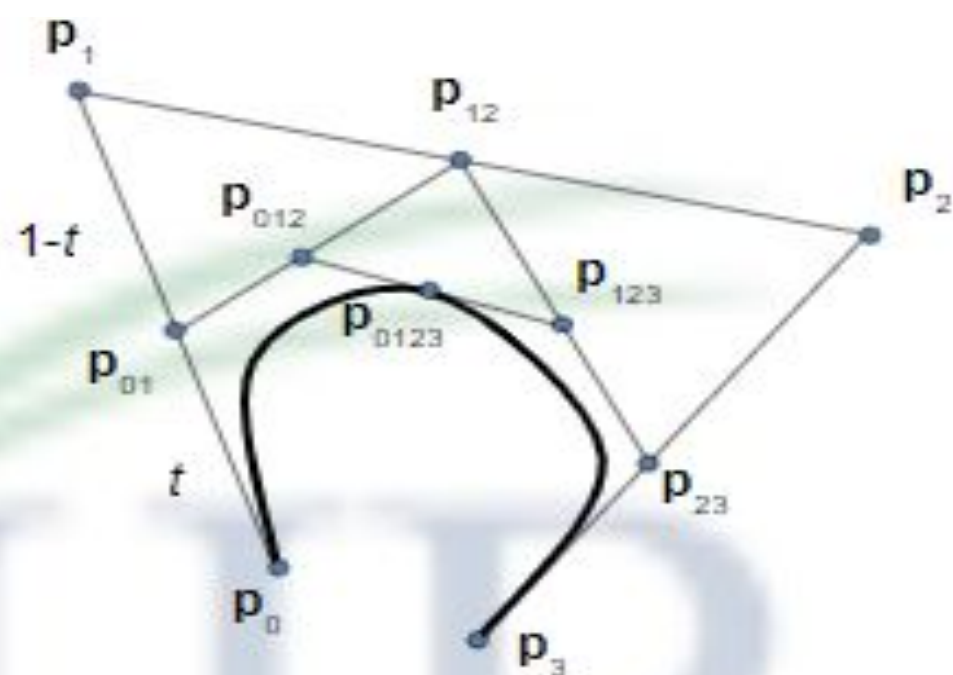
Cubic Bezier Curve

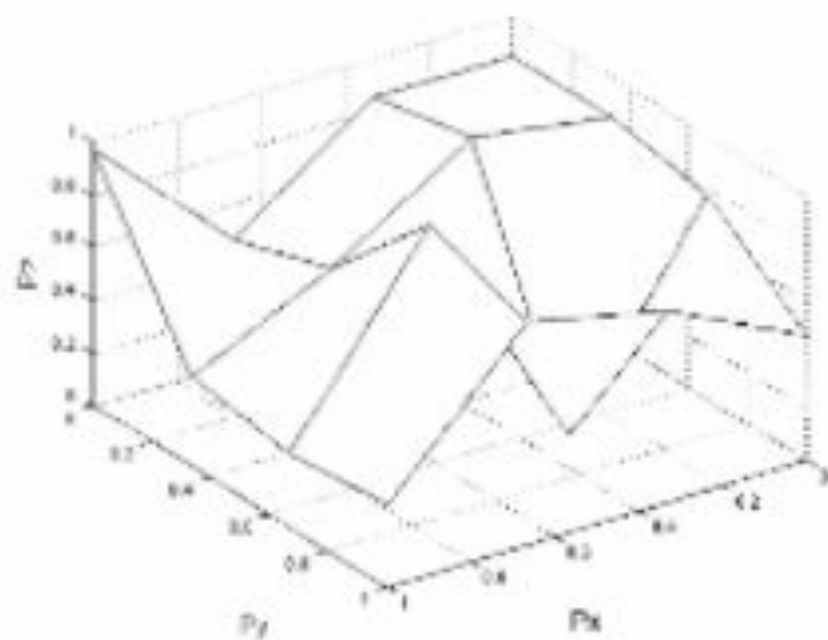
BEZIER CURVE

- In Cubic Bezier curve require 4 sample points
- The curve begins at 1st sample point and end at 4th sample point

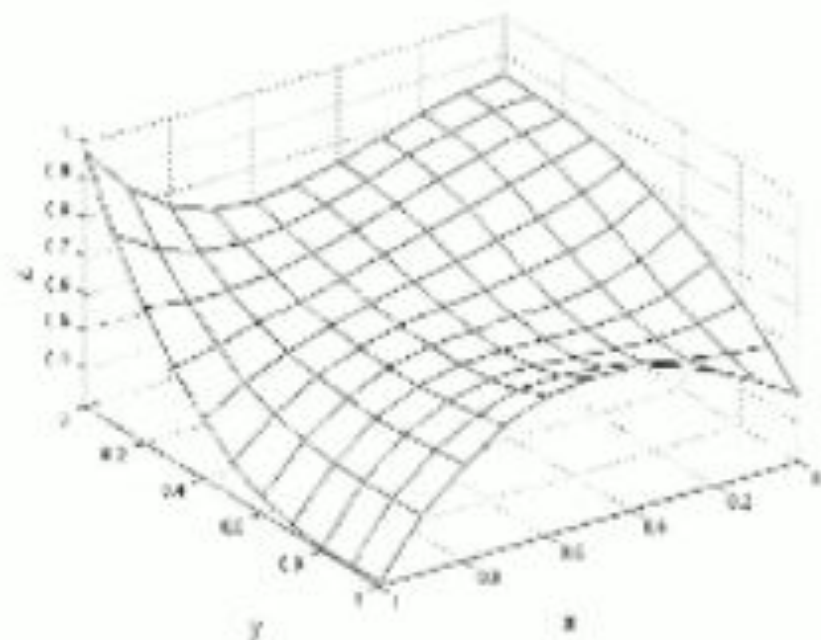
$$\mathbf{p}_{01} = (1-t)^3 \mathbf{p}_0 + 3t(1-t)^2 \mathbf{p}_1 + 3t^2(1-t) \mathbf{p}_2 + t^3 \mathbf{p}_3$$

- $$\mathbf{p}_{01} = (1-t) \mathbf{p}_0 + t \mathbf{p}_1$$
- $$\mathbf{p}_{12} = (1-t) \mathbf{p}_1 + t \mathbf{p}_2$$
- $$\mathbf{p}_{23} = (1-t) \mathbf{p}_2 + t \mathbf{p}_3$$
- $$\mathbf{p}_{012} = (1-t) \mathbf{p}_{01} + t \mathbf{p}_{12}$$
- $$\mathbf{p}_{123} = (1-t) \mathbf{p}_{12} + t \mathbf{p}_{23}$$
- $$\mathbf{p}_{0123} = (1-t) \mathbf{p}_{012} + t \mathbf{p}_{123}$$
- Subdivides curve at $\mathbf{p}_{0123}$
 - $\mathbf{p}_0 \mathbf{p}_{01} \mathbf{p}_{012} \mathbf{p}_{0123}$
 - $\mathbf{p}_{0123} \mathbf{p}_{123} \mathbf{p}_{23} \mathbf{p}_3$
 - Repeated subdivision converges to curve



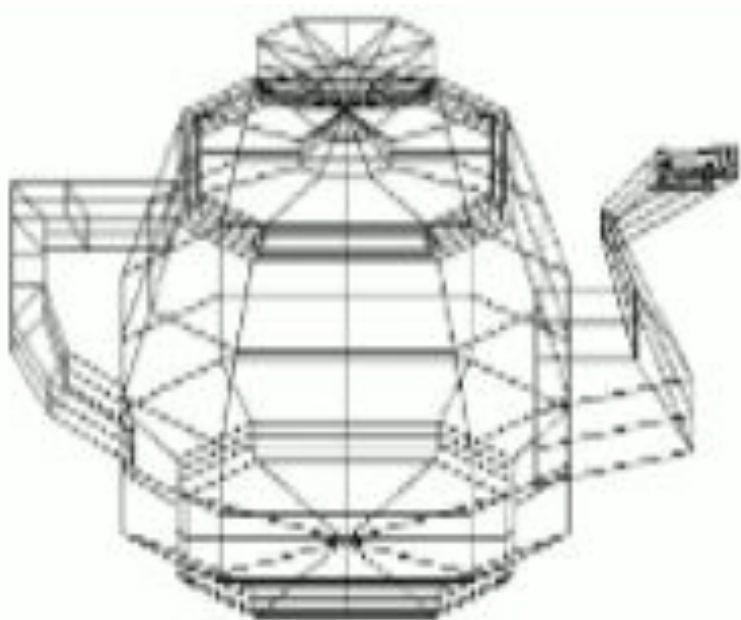


a control points

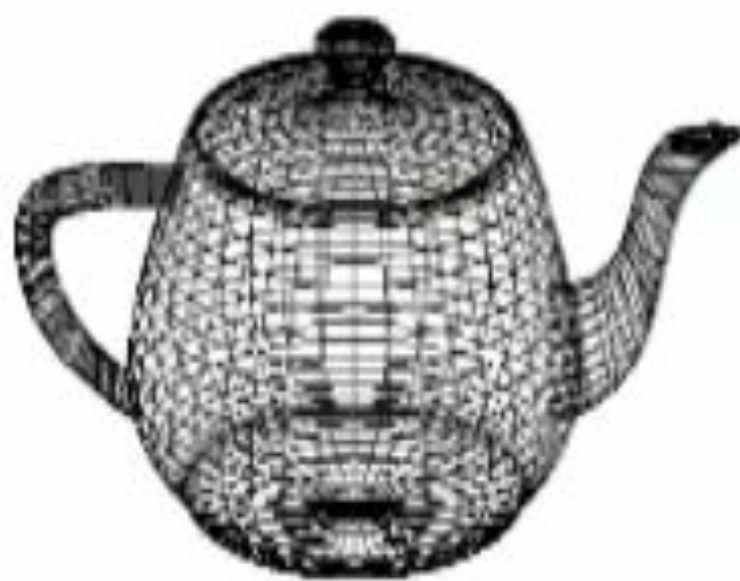


b Bézier surface

Figure : Bézier surface generated using random control points.



a control points



b Bézier surfaces

Figure : The Utah teapot control points and with Bézier surfaces applied.